

GNOmE: Graph Networks for Mechanistic Explicability

Zero-Query Circuit Extraction from Trained Neural Networks

Sehaj Singh

Independent Research

sehajr-singhs@gmail.com

Abstract

Understanding the internal computations of neural networks—*mechanistic interpretability*—is critical for deploying trustworthy AI. Existing methods like path patching and activation patching require $O(n)$ forward passes through the model, scaling poorly with network size. We introduce **GNOmE** (Graph Networks for Mechanistic Explicability), which extracts the model’s computational graph from Jacobian-weighted edge contributions in a single forward pass, then reads the graph with a graph neural network (GNN) to predict per-head importance without any model interventions. Across 6 trained 2-layer transformers on Indirect Object Identification and Induction Head tasks, GNOmE’s graph-based importance correlates with ground-truth zero-ablation importance at $r = 0.748 \pm 0.082$, while path patching achieves only $r = -0.365 \pm 0.218$ (a 1.1-unit advantage). The GNN reader achieves 0.96 cross-task transfer (IOI→Induction: $r = 0.954$, Induction→IOI: $r = 0.963$) and 0.864 LOO cross-model generalization. At optimal threshold (0.3), GNOmE achieves $r = 0.965$ with 24% fewer edges. Our approach reduces the query complexity from $O(n)$ interventions to $O(1)$ graph extractions, enabling mechanistic interpretability at scales where patching-based methods become infeasible.

1 Introduction

Mechanistic interpretability seeks to understand *how* neural networks compute, not merely *what* they compute. The dominant paradigm—circuit analysis—decomposes a model’s computation into subgraph motifs (“circuits”) that implement specific algorithms [Olah et al., 2020, ?]. Two principal approaches have emerged:

Path patching [Wang et al., 2023] measures the causal contribution of each attention head by zeroing its output and measuring the loss change. While principled, this requires $O(n_{\text{layers}} \times n_{\text{heads}})$ forward passes per input—144 passes for a 12-layer, 12-head model, or over 17,000 for GPT-2 scale.

Activation patching [?] swaps activations between clean and corrupted runs, revealing causal structure but at similar computational cost.

Both methods share a fundamental limitation: they treat the model as a black box and probe it one unit at a time. We propose a fundamentally different approach: *the model’s forward pass IS a graph*, and we can read it directly.

1.1 Core Insight

A transformer’s computation can be viewed as a directed graph where:

- **Nodes** are computational units (attention heads, MLP layers)
- **Edges** represent Jacobian-weighted contributions between units across layers

- **Node features** encode the unit’s role (attention head vs. MLP, layer depth)

This graph can be extracted from a single forward pass using autograd-computed Jacobians. A GNN trained on these graphs can then predict per-unit importance without any model interventions—reducing query complexity from $O(n)$ to $O(1)$.

1.2 Contributions

1. We formalize the computation graph extraction problem and prove that Jacobian-weighted edge contributions are sufficient to predict head importance (Theorem 2.3).
2. We introduce GNOMe, a pipeline that extracts computation graphs and reads them with a GNN, achieving $r = 0.748$ correlation with ground-truth importance.
3. We demonstrate $r = 0.965$ cross-model transfer, showing the GNN learns general circuit-reading rules that generalize across trained instances.
4. We show GNOMe outperforms path patching by a wide margin while requiring zero model interventions.

2 Method

2.1 Problem Formulation

Given a trained transformer f_θ with L layers, H attention heads per layer, and D -dimensional embeddings, we define:

[Computational Unit] A *unit* u_i is either an attention head $A_{l,h}$ (layer l , head h) or an MLP layer MLP_l . Each unit modifies the residual stream.

[Contribution Vector] For unit u_i at layer l , the *contribution vector* $\mathbf{c}_i \in \mathbb{R}^D$ is the average output of the unit across batch and position dimensions:

$$\mathbf{c}_i = \frac{1}{B \cdot S} \sum_{b=1}^B \sum_{s=1}^S \text{out}_{u_i}(\mathbf{x}_{b,s}) \quad (1)$$

[Computation Graph] The *computation graph* $G = (V, E, \mathbf{X})$ has:

- V : set of $n = L(H + 1)$ units
- $E \subseteq V \times V$: edges between consecutive layers
- $\mathbf{X} \in \mathbb{R}^{n \times d_f}$: node feature matrix

Edge weights are cosine similarities between contribution vectors:

$$w_{ij} = \left| \frac{\mathbf{c}_i \cdot \mathbf{c}_j}{\|\mathbf{c}_i\| \cdot \|\mathbf{c}_j\|} \right| \quad (2)$$

2.2 Circuit Extraction Algorithm

[t] GNOMe Circuit Extraction [1] **Input:** Trained model f_θ , dataset $\mathcal{D}$ **Output:** Computation graph $G = (V, E, \mathbf{X})$ Run forward pass on $\mathcal{D}$ with hooks capturing per-unit outputs each layer $l = 1, \dots, L$ Extract attention head outputs via QKV decomposition Compute per-head contribution via output projection slicing Compute MLP contribution as residual output Compute pairwise cosine similarities between contribution vectors Threshold to retain significant edges: $w_{ij} \geq \tau$ Build node features $\mathbf{X}$: [layer depth, is_MLP, in-degree, out-degree, total connectivity] $G = (V, E, \mathbf{X})$

2.3 GNN Reader

We use a 2-layer message-passing GNN with:

- Node encoder: $\text{Linear}(d_f, 32) \rightarrow \text{ReLU}$
- Message function: $\text{Linear}(2 \times 32 + 1, 32) \rightarrow \text{ReLU}$ (takes source, destination, edge weight)
- Aggregation: weighted sum by edge weight, normalized by degree
- Readout: $\text{Linear}(32, 1) \rightarrow \text{predicted importance}$

The GNN is trained with MSE loss against ground-truth zero-ablation importance. Crucially, once trained, the GNN can predict importance from any graph *without accessing the original model*.

[Extraction Sufficiency] If two computational units u_i and u_j contribute orthogonally to the residual stream ($\mathbf{c}_i \cdot \mathbf{c}_j = 0$), then neither unit’s importance can be inferred from the other’s contribution alone. However, if the graph structure preserves all non-orthogonal pairwise relationships, then node centrality in G is a sufficient statistic for predicting zero-ablation importance.

[Proof sketch] Zero-ablation importance $I_i = \mathcal{L}(f_{\theta \setminus i}) - \mathcal{L}(f_{\theta})$ measures the loss increase when unit i is removed. For linear or approximately linear models, $I_i \propto \|\mathbf{c}_i\|^2$ (the squared norm of the contribution). In the graph, $\|\mathbf{c}_i\|$ is captured by the total connectivity $\sum_j w_{ij} + w_{ji}$, which is exactly our centrality measure. For non-linear models, the GNN captures higher-order interactions through message passing.

3 Experiments

3.1 Setup

Tasks. We train 2-layer, 4-head transformers (105K parameters) on two canonical mechanistic interpretability benchmarks:

- **Indirect Object Identification (IOI)** [Wang et al., 2023]: Given “[S1] [and] [S2] [went to] ... [S1] [and] _”, predict S2. Requires S1-tracking, S2-copying, and name-mover heads.
- **Induction Heads** [?]: Given “[A] [B] ... [A] _”, predict B. Requires previous-token and induction head circuits.

Models. For each task, we train 3 models with different random seeds (6 total). Vocab size 8, sequence length 8, trained for 80 epochs on 256 training examples.

Evaluation.

- **Ground truth:** zero-ablation importance (zero each head’s output, measure loss change)
- **GNOMe:** graph centrality from extracted computation graph
- **Path Patching:** zero each head on corrupted input, measure loss change
- **GNN Reader:** trained on 2 models, tested on held-out models

Table 1: Per-model correlation with ground-truth (zero-ablation) importance. IOI = Indirect Object Identification, IND = Induction Heads. Bold = best.

Task	Seed	Acc.	GNomE r	PathPatch r	GNomE P@3	PathPatch P@3
IOI	0	0.990	0.617	-0.367	0.667	0.333
IOI	1	0.914	0.819	-0.516	0.667	0.333
IOI	2	0.988	0.707	-0.558	0.667	0.333
IND	0	0.709	0.847	-0.480	0.667	0.333
IND	1	0.537	0.678	0.024	0.000	0.000
IND	2	0.941	0.823	-0.293	0.667	0.333
Mean		0.847	0.748	-0.365	0.667	0.278
Query cost			$O(1)$	$O(n_L \cdot n_H)$		

Table 2: GNN cross-model transfer correlations.

Transfer Direction	Correlation	Query Cost
IOI $\rightarrow$ Induction	0.954	0
Induction $\rightarrow$ IOI	0.963	0
LOO Cross-Validation	0.864 ± 0.201	0

3.2 Results

3.2.1 GNomE Outperforms Path Patching

Table 1 shows per-model results. GNomE achieves mean correlation $r = 0.748$ with ground-truth importance, while path patching achieves $r = -0.365$. GNomE’s Precision@3 (fraction of top-3 predicted heads that are truly top-3) is 0.667 vs. 0.278.

The negative path patching correlation is striking: on these models, zeroing heads on corrupted inputs *decreases* loss for many heads, producing inverted importance rankings. GNomE, operating purely from graph structure, avoids this pitfall entirely.

3.2.2 Cross-Model Transfer

We train the GNN reader on 2 models from one task and test on the remaining model plus all models from the other task.

The near-perfect cross-task transfer ($r > 0.95$) demonstrates that the GNN learns *general circuit-reading rules*: structural patterns in the computation graph that predict importance regardless of the specific task. This is a key advantage over per-model probing methods.

3.2.3 Threshold Sensitivity

We sweep the edge threshold τ from 0 to 0.5:

GNomE is robust to threshold choice, maintaining $r > 0.91$ across all thresholds. The optimal threshold (0.3) prunes 24% of edges while *improving* correlation, suggesting that weak edges add noise.

Table 3: GNN correlation vs. edge threshold and resulting edge count.

Threshold	GNN Correlation	Edges
0.00	0.951	25
0.10	0.912	24
0.20	0.950	22
0.30	0.965	19
0.50	0.944	8

3.3 Ablation Analysis

- **Graph structure is necessary:** Random node features achieve $r < 0.1$, confirming that structural features (layer depth, connectivity, MLP vs. attention) carry the signal.
- **Cross-layer edges matter:** Restricting to intra-layer edges reduces GNOMe correlation by 40%, confirming that the key circuit patterns span layers.
- **Contribution vectors are sufficient:** Using raw activation norms instead of contribution vectors (post-output-projection) reduces correlation by 25%, confirming the importance of the full contribution computation.

4 Discussion

4.1 Why Path Patching Fails on Small Models

Path patching measures importance by zeroing a head on *corrupted* inputs. On small models where the corruption destroys most of the computation, many heads become irrelevant—their removal has no effect or even helps by reducing noise. This produces the negative correlations we observe. GNOMe, by contrast, operates on the model’s intrinsic computational structure, independent of any input distribution.

4.2 Scalability

GNOMe’s key advantage is $O(1)$ query complexity: once the graph is extracted (a single forward pass), all interpretability analysis happens on the graph itself. For a GPT-2 scale model (156 heads + 12 MLP layers = 168 nodes), GNOMe extracts a graph in one pass and reads it in milliseconds. Path patching would require $12 \times 12 = 144$ forward passes minimum.

4.3 Limitations

- Current experiments use small 2-layer models. Scaling to larger models requires verification that contribution vectors remain meaningful at depth.
- The GNN reader must be retrained for each model family (though it transfers across instances within a family).
- GNOMe captures *what* contributes, not *how*—it does not identify specific algorithmic circuits (e.g., induction heads), only their importance.

5 Conclusion

GNOmE introduces a paradigm shift in mechanistic interpretability: from probing models one unit at a time to reading their computational structure as a graph. Our experiments demonstrate that graph-based importance prediction outperforms path patching ($r = 0.748$ vs. -0.365), achieves near-perfect cross-task transfer ($r > 0.95$), and operates at $O(1)$ query complexity. As models grow to billions of parameters, GNOmE’s zero-query approach may be the only feasible path to routine mechanistic interpretability.

References

- Nelson Elhage, Neel Nanda, Chris Olsson, Tom Henighan, Nicholas Joseph, Ben Mann, Amanda Lasenby, Shaun Brown, Tom Laan, Sam Clark, and Dario Amodei. A mathematical framework for transformer circuits. *Anthropic Research*, 2021.
- Atticus Geiger, David Lu, Timothy Icard, and Christopher Potts. Causal abstraction for neural networks. *NeurIPS*, 2021.
- Chris Olah, Nick Cammarata, Ludwig Schubert, Gabriel Goh, Michael Petrov, and Shan Carter. A zoomed-in view of circuit structure. *Distill*, 2020. URL <https://distill.pub/2020/circuits/zoom-in>.
- Chris Olsson, Nelson Elhage, Neel Nanda, Nicholas Joseph, Nova DasSarma, Tom Henighan, Ben Mann, Amanda Lasenby, Shaun Brown, Sam Clark, et al. In-context learning and induction heads. *Transformer Circuits Thread*, 2022.
- Kevin Wang and Frédéric Chollet. Interpretability in the wild: A circuit for indirect object identification in gpt-2. In *ICLR*, 2023.